

BỘ CÂU HỎI PHẢN BIỆN DỰ KIẾN VÀ ĐÁP ÁN

Đồ án tốt nghiệp HK253-DATN-055 — Xây dựng giải pháp gom cụm tình trạng giao thông ở TP.HCM

Sinh viên: Trần Quang Huy — 2211288 (tài liệu ôn tập cá nhân, không nộp kèm báo cáo)

Mẹo trả lời:¹ trả lời theo khung *trực tiếp* → *căn cứ số liệu trong báo cáo* → *hạn chế/nếu bị hỏi sâu*. Thời lượng mỗi câu 45–90 giây; với câu chưa chắc, thừa nhận giới hạn rồi chỉ hướng phát triển thay vì babbling.

Phần A — Dữ liệu và mô hình gom cụm

Câu A1. Vì sao chọn gom cụm (không giám sát) mà không dùng phân loại có giám sát, khi bài toán rõ ràng là gán nhãn trạng thái giao thông?

Trả lời: Vì không có nhãn sẵn: gán nhãn “tắc/trung bình/thoáng” thủ công cho hàng nghìn đoạn đường là bất khả thi và thiếu khách quan; ranh giới trạng thái còn mờ và khác nhau giữa các đô thị. Gom cụm cho phép hệ thống tự khám phá trạng thái từ dữ liệu, sau đó diễn giải ngược từng cụm thành trạng thái có ý nghĩa. Nhóm trạng thái tham chiếu (LOS + tốc độ tương đối) chỉ dùng để *đánh giá ngoại bộ*, không dùng để huấn luyện — tránh vòng lặp tự xác nhận.

Câu A2. Vì sao chọn $k = 6$? Căn cứ nào?

Trả lời: Ba căn cứ: (1) đồ thị Elbow trên khoảng $k = 2..12$ gãy rõ quanh 5–6; (2) tính diễn giải — 6 cụm tương ứng 6 trạng thái giao thông có ý nghĩa thực tiễn (kẹt nặng nhất → thoáng hoàn toàn), mỗi cụm có profile congestionIndex trung bình phân tách rõ (0,62 đến 1,0); (3) so sánh đa chỉ số cho thấy $k = 6$ cân bằng Silhouette/DBI/CH tốt nhất trên nhánh Feature Selection. GMM dùng thêm BIC/AIC và DBSCAN dùng k-distance graph để chọn tham số tương ứng.

Câu A3. Silhouette 0,4183 là mức thấp so với các bài toán kinh điển (iris 0,5+). Anh chấp nhận thế nào?

Trả lời: Dữ liệu giao thông thật có ranh giới trạng thái mờ tự nhiên (nhiềuDetector đoạn nằm giữa hai trạng thái), nên Silhouette luôn thấp hơn dữ liệu tổng hợp phân tách rõ. Quan trọng hơn là so sánh tương đối: cùng một dữ liệu, nhánh 40 thuộc tính chỉ đạt 0,1310 — Feature Selection cải thiện gấp 3,2 lần; và các cụm sau phân loại có thứ hạng nghiêm trọng nhất quán với trực giác giao thông (đã xác minh bằng profile từng cụm). Trong bài toán thực, tính diễn giải và nhất quán cụm quan trọng hơn con số tuyệt đối.

Câu A4. Vì sao DBSCAN kém nhất (Silhouette âm)?

Trả lời: Vì mật độDetector đoạn giao thông không đồng đều: nội đô dày đặc, ngoại thành

¹Ghi chú dùng nội bộ.

thừa — một bán kính ϵ duy nhất không thể thỏa cả hai vùng, dẫn tới rất ít cụm lớn và lượng nhiễu lớn (Calinski–Harabasz chỉ 2.838 so với 147.575 của K-Means). Đây là hạn chế đã biết của DBSCAN trên dữ liệu mật độ không đều; hướng khắc phục là DBSCAN biến thể (HDBSCAN) cho phép mật độ thay đổi — ghi nhận ở hướng phát triển.

Câu A5. Trong mô hình dự đoán tốc độ, làm sao tránh leakage?

Trả lời: currentSpeed là biến đích, còn congestionIndex và jamFactor đều được suy ra trực tiếp từ currentSpeed nên bị loại khỏi đầu vào. Đầu vào chỉ gồm hạ tầng (làn, giới hạn tốc độ, độ cong, độ dốc, giao lộ), thời gian (giờ, thứ, loại ngày) và thời tiết. Nhờ vậy MAE 2,19 km/h (bản cloud-lite 60 cây vừa RAM 512MB của Render; bản đầy đủ 120 cây đạt 1,84 km/h nhưng chiếm 330MB RAM) đo trên 7.452 mẫu giữ-out, so baseline 8,25 km/h (luôn giả định đường thoáng).

Câu A6. Thời tiết chỉ có theo ngày — có đủ tinh không?

Trả lời: Đúng là hạn chế: precipitation/temperature là giá trị ngày, không phản ánh cơn mưa cục bộ 15 phút. Tuy nhiên (1) 100% mẫu có dữ liệu thời tiết nhờ join Open-Meteo Archive theo đúng khoảng thời gian snapshot, (2) mưa lớn cả ngày vẫn được phản ánh, và (3) hướng phát triển đã ghi nhận: dùng Open-Meteo API theo giờ để nâng độ tinh.

Câu A7. Vì sao không dùng LSTM/Graph Neural Network cho dự báo?

Trả lời: LSTM/GNN đòi hỏi chuỗi thời gian liên tục theo từngDetector đoạn — dữ liệu hiện tại là các snapshot rời rạc không đều chu kỳ, chưa đủ cấu trúc cho mô hình dãy. Đánh giá nhanh bằng RF trên đặc trưng đa nguồn đã cho MAE 2,19 km/h (R^2 0,919) — đủ cho nghiệp vụ cảnh báo theoDetector đoạn. GNN được ghi nhận ưu tiên số một ở hướng phát triển khi hệ thống chuyển sang thu thập streaming liên tục.

Phần B — Kiến trúc và tích hợp

Câu B1. Mô hình chạy trên điện thoại hay trên server? Vì sao chia như vậy?

Trả lời: Cả hai, theo kiến trúc lai: mô hình gom cụm K-Means xuất JSON và nhúng trong app (assets) để tô màu bản đồ — suy luận tức thì, không cần mạng, chi phí chỉ một phép nhân vector nhỏ; mô hình Random Forest dự đoán tốc độ chạy trên server vì công kênh hơn và cần tái huấn luyện thường xuyên. Điểm mấu chốt là *cùng một tệp cluster_model.json* chạy ở cả pipeline, server và app — mộtDetector đoạn luôn về cùng một cụm bất kể nơi suy luận.

Câu B2. Nếu server chết, app có sập không?

Trả lời: Không. Mọi lời gọi ML đều bất đồng bộ, timeout ngắn (1,5s kết nối / 4s đọc), thử lần lượt vài địa chỉ ứng viên; server không phản hồi thì fallback im lặng về logic on-device (chấm điểm tuyến theo congestionIndex live) và chip trạng thái chuyển “Server ML: ngoại tuyến”. Tất cả tính năng cốt lõi (bản đồ, chỉ đường, tin tức) không phụ thuộc server. Điều

này được kiểm thử ở ca TC06.

Câu B3. Dữ liệu live lấy như nào và tần suất bao nhiêu? Có đót quota không?

Trả lời: Hai đường: (1) app gọi TomTom Flow *theo khu vực đang xem* — chỉ khi zoom ≥ 14 , tối đa 150Detector đoạn/lần render, cache TTL 3 phút, ưu tiênDetector đoạn gần tâm màn hình; (2) server có bộ refresher quét 50Detector đoạn giám sát mỗi 15 phút. Tổng tiêu thụ ≈ 4.800 lời gọi/ngày chia 8 khóa ≈ 600 /khóa/ngày, an toàn trong hạn mức free 2.500/khóa/ngày. Khi hết quota, khóa tự xoay vòng vàDetector đoạn thiếu dữ liệu tô xám thay vì suy đoán.

Câu B4. Khoảng 15 phút/lần cập nhật phía server có đủ “realtime”?

Trả lời: Cần phân biệt hai lớp: lớp *đang xem* của app là realtime theo yêu cầu (mỗi lần pan/zoom là fetch mới, TTL 3 phút); lớp refresher 15 phút của server chỉ là *bức tranh nền* phục vụ live-status và thống kê. Giao thông đô thị thay đổi trên thang hàng chục phút nên 15 phút là hợp lý cho nền; nếu cần tinh hơn, chỉ cần hạ biến môi trường TRAFFIC_REFRESH_INTERVAL — chấp nhận tăng quota.

Câu B5. Vì sao dùng FastAPI thay vì Node.js/Express như đề án gốc?

Trả lời: Ba lý do: (1) FastAPI chạy *cùng tiến trình* với mô hình scikit-learn — bỏ hẳn lớp mạng trung gian Node→Python, giảm độ trễ và điểm lỗi; (2) xác thực schema tự động bằng Pydantic và Swagger UI tự sinh; (3) giữ nguyên *giao diện API* đã thiết kế trong báo cáo (7 endpoint /api/ml/*) nên về mặt hợp đồng dịch vụ không đổi so với đề án gốc.

Câu B6. Lỗi “Permission denied” với Firebase là gì? Sửa thế nào?

Trả lời: Đó là quy tắc bảo mật của Realtime Database đang khóa toàn bộ (kể cả người dùng đã xác thực) — mình xác nhận bằng REST: đăng nhập lấy idToken thành công nhưng đọc Users vẫn 401. Cách sửa: công bố rules cho phép Users/{uid} đọc/ghi khi auth.uid == {uid} và incidents đọc/ghi khi auth != null (chi tiết Bảng 5.6 báo cáo). Lưu ý Firebase *không chết*: Auth hoạt động bình thường, chỉ mỗi rules cần mở đúng phạm vi.

Câu B7. Mô hình K-Means nhúng JSON bảo mật thế nào? Ai có thể sửa nó?

Trả lời: Hiện mô hình nằm trong assets của APK và models/ của server — ai giải nén APK cũng đọc được tâm cụm. Đây là dữ liệu *không nhạy cảm* (tham số mô hình), rủi ro lớn nhất là bị thay thế bằng mô hình giả. Phương án tăng cường đã ghi nhận: ký APK, kiểm tra checksum của tệp mô hình lúc khởi động, và dài hạn là phục vụ mô hình qua server có chữ ký.

Phần C — Ứng dụng và trải nghiệm

Câu C1. Cụ thể gom cụm mang lại gì mà ngưỡng màu đơn giản không có?

Trả lời: Ba điểm: (1) *ngữ nghĩa* — mỗi màu tương ứng một trạng thái có profile rõ (đoạn đỏ thuộc cụm có congestionIndex trung bình 0,62, tức “vùng kẹt điển hình”), không phải ngưỡng cục bộ tùy cơ hội; (2) *nhất quán* — cùng một mô hình chạy trên app và server nên nhận trạng thái thống nhất toàn hệ thống; (3) *nền tảng* — cụm là đầu vào cho định tuyến né kẹt, chấm điểm tuyến và dự báo ML, thay vì ba logic ngưỡng rời rạc.

Câu C2. Chứng minh cụ thể mộtDetector đoạn được phân loại như thế nào?

Trả lời: Ví dụ có trong Bảng 6.3–6.4 báo cáo:Detector đoạn 1,2 km chạy 25/60 km/h cho 9 đặc trưng (speedLimitRatio 0,417; congestionIndex 0,417; occupancy 58,3%...); sau chuẩn hóa, khoảng cách tới sáu tâm lần lượt là 7,72/5,65/3,56/8,32/10,17/5,14 — gần tâm cụm 2 nhất, thứ hạng 0, tô đỏ đậm. Đối chiếuDetector đoạn 52/55 km/h về cụm 0, tô xanh. Suy luận chỉ tốn vài trăm phép nhân số thực.

Câu C3. Vì sao hệ số phạt kẹt khi định tuyến tối đa là 1,6, không cao hơn?

Trả lời: Đã cân bằng bằng thực nghiệm thủ công: phạt quá nhẹ thì thuật toán vẫn chọnDetector đoạn kẹt ngắn; phạt quá nặng (vd. 5x) dẫn tới chọn đường vòng vô lý xuyên hẻm. 1,6 nghĩa làDetector đoạn kẹt nhất “đắt” hơn 60% thời gian thực — đủ để ưu tiênDetector đoạn thoáng mà không phá tính khả thi của tuyến. Đây là tham số có thể tinh chỉnh theo phản hồi người dùng.

Câu C4. Sức khỏe tuyến A–F tính như thế nào và tin được không?

Trả lời: Lấy ~40 điểm mẫu dọc tuyến, mỗi điểm traDetector đoạn gần nhất và đọc congestionIndex live (đã fetch sẵn); phân loại kẹt nặng/trung bình/ thoáng rồi quy đổi: delay ước tính = $(tlktnng \times 3 + tltrungbnh) \times 10$ phút, điểm chữ theo tỉ lệ. Chỉ tính mẫu có dữ liệu thật; mẫu thiếu dữ liệu ghi NO_DATA riêng. Giới hạn được thừa nhận: chất lượng phụ thuộc độ phủ TomTom tạiDetector đoạn đó.

Câu C5. Ứng dụng khác biệt gì so với Google Maps?

Trả lời: Google Maps tốt ở độ phủ đội xe nhưng: (1) hiển thị trạng thái theo ngưỡng màu cố định, không phân cụm theo đặc thù TP.HCM; (2) không cho người dùng thấy “đoạn này thuộc trạng thái nào, dự báo bao nhiêu”; (3) không mở API phân tích. TraffiGo ngược lại: trạng thái phân cụm trên dữ liệu TP.HCM, minh bạch cụm/dự báo/rủi ro ngay trênDetector đoạn, và có API ML mở. Đổi lại độ phủ live của mình thinner — đã ghi nhận trung thực ở Chương 7.

Phần D — Kiểm thử và vận hành

Câu D1. Kiểm thử như thế nào để tin được sản phẩm chạy đúng?

Trả lời: Ba tầng: (1) *unit test* JVM cho các thành phần thuần logic (phân cụm on-device, chấm điểm tuyến, PathFinder) nạp đúng tệp mô hình thật; (2) *kiểm thử chức năng* 18 ca trên trình giả lập Android 15 phủ hết 14 yêu cầu chức năng; (3) *kiểm thử hồi quy* — mỗi lỗi phát hiện được sửa thì chạy lại các ca liên quan ở nhiều cấu hình (server on/off, nhiều mức zoom). Đo hiệu năng server: dự đoán 67 ms trung bình.

Câu D2. Kể một lỗi khó nhất anh gặp và cách xử lý?

Trả lời: Lỗi crash “Not on the main thread”: khi tối ưu thứ tự fetch TomTom, mình gọi `getCameraPosition()` từ luồng nền để sắp thứ tựDetector đoạn theo khoảng cách tới tâm màn hình — Maps SDK cấm tuyệt đối. Bài học: ràng buộc luồng của SDK bên thứ ba phải được rà khi chuyển logic sang background; sửa bằng cách chụp tâm camera ở main thread rồi truyền giá trị xuống. Quá trình này phát hiện tổng cộng ~18 lỗi (crash, ANR, rò rỉ trạng thái) và đều đã kiểm thử hồi quy.

Câu D3. Nếu 1.000 người dùng đồng thời thì hệ thống chịu được không?

Trả lời: Khác biệt quan trọng: gom cụm và tô màu chạy *trên thiết bị người dùng* nên tải không dồn về server; server chỉ xử lý dự đoán — mỗi request 66 ms, Uvicorn async có thể song song hóa; 1.000 người dùng không đều gọi API (chỉ khi bấm vàoDetector đoạn/tìm đường). Nút thắt thật sự là quota TomTom phía server và xác thực API chưa có — đã ghi nhận ở rủi ro triển khai (Bảng 8.1) cùng phương án: rate-limit, token, cache theo vùng.

Câu D4. Dự án này dùng được cho thành phố khác không?

Trả lời: Kiến trúc và pipeline dùng được nguyên vẹn (chỉ đổi bounding box vàDetector đoạn nguồn); còn *mô hình* cần huấn luyện lại vì trạng thái giao thông là đặc thù từng đô thị (thói quen, hỗn hợp phương tiện, hạ tầng) — đúng triết lý “gom cụm trên dữ liệu của chính thành phố đó”. Đó là lý do mô hình để dạng JSON tái huấn luyện một lệnh thay vì hardcoded.

Câu D5. Phần cứng/tiện ích gì em tự làm, gì dùng của người khác?

Trả lời: Tự làm: pipeline thu thập/làm sạch/feature selection, toàn bộ đánh giá mô hình, export JSON, server FastAPI với 7 endpoint + refresher, toàn bộ ứng dụng TraffiGo (~8.400 dòng Java, 20 màn hình) và tích hợp hai chiều. Dùng của bên thứ ba theo API công khai: dữ liệu TomTom/OSM/Mapbox/Open-Meteo, Maps SDK, Firebase, mô hình Gemini cho trợ lý giọng nói. K-Means/RF dùng scikit-learn (chuẩn ngành), phần *su* là thiết kế thí nghiệm và diễn giải.

Câu D6. Nếu chấm lại, điểm yếu lớn nhất của đề án là gì?

Trả lời: Trung thực nhất: (1) gom cụm là pipeline offline — chưa streaming; (2) dữ liệu chỉ

~5 tuần nên chưa khái quát theo mùa; (3) API chưa có xác thực/rate-limit cho môi trường công khai; (4) Weather chỉ theo ngày. Cả bốn đều đã ghi nhận rõ ở Chương 7–8 kèm hướng xử lý cụ thể (streaming clustering, Open-Meteo theo giờ, token + rate-limit, thu thập kéo dài). Em ưu tiên xử lý theo thứ tự tác động: xác thực API trước khi công khai, rồi streaming.

Lưu ý: câu trả lời trên bám số liệu thật trong báo cáo (Silhouette 0,4183; MAE 1,84 km/h; R^2 0,941; 149.023 mẫu; 67 ms/request...). Nếu hội đồng hỏi số khác, trả lời đúng con số trong báo cáo và chỉ trang/mục tương ứng.